Stringjet në Python

Alion Emini

Kurs i Programimit në Python
kurse.informatika
November 6, 2025

Contents

1 Stringjet në Python **5**

1.1 Aksesimi i karaktereve individualë në një string . . . 5

1.1.1 Iterimi mbi një string me ciklin `for` . . . 6

1.2 Indeksimi . . . 7

1.2.1 Përjashtimet `IndexError` . . . 9

1.3 Funksioni `len` . . . 9

1.4 Konkatenimi i stringjeve . . . 9

1.5 Stringjet janë *immutable* . . . 11

1.6 Prerja (*slicing*) e stringjeve . . . 12

1.7 Testimi, kërkimi dhe manipulimi i stringjeve . . . 15

1.7.1 Testimi i stringjeve me `in` dhe `not in` . . . 15

1.7.2 Metoda testimi për stringje . . . 15

1.7.3 Metoda modifikimi (kthejnë kopje të modifikuara) . . . 17

1.7.4 Kërkimi dhe zëvendësimi . . . 19

1.8 Operatori i përsëritjes (`*`) . . . 20

1.9 Ndarja e një stringu me `split()` . . . 21

1.10 Karakteret e *escape*-it në stringje . . . 22

1.10.1 Pse duhen karakteret e *escape*-it . . . 22

1.10.2 Shpesh të përdorura . . . 22

1.10.3 Shembuj të shpejtë . . . 23

1.11 Formatimi i stringjeve . . . 23

1.11.1 F-strings (rekomanduar) . . . 23

1.11.2 `str.format()` . . . 23

1.11.3 Operatori i vjetër `%` (stil "printf") . . . 24

Chapter 1

Stringjet në Python

Objektivat e kapitullit

Tema	Çfarë do të dish pas këtij kapitulli
Aksesimi i karaktereve	Iterim me `for`, indekse pozitive/negative dhe kur ndodh `IndexError`.
Funksioni `len()`	Përdorimi i `len` për kufij të sigurt në cikle dhe llogaritjen e gjatësisë.
Konkatenimi	Operatorët `+` dhe `+=`; çfarë krijohet realisht (string i ri).
Immutability	Pse `string[i] = 'x'` s'lejohet dhe pasojat praktike.
Prerja (slicing)	Sintaksa `start:end[:step]`, indekse negative dhe rastet kufi.
Testim/Kërkim	`in` / `not in`, `find`, `startswith`, `endswith`.
Modifikim pa anësore	`lower`, `upper`, `strip/lstrip/rstrip`, `replace`.
Përsëritja	Operatori `*` dhe skenarë tipikë (separatorë, ASCII-art).
Ndarja e tekstit	`split([sep][, maxsplit])` dhe pastrim me `strip()`.
Escape characters	`\n`, `\t`, `\"`, `\\`, `\xhh`, `\ooo`.
Formatimi i stringjeve	f-strings, `str.format()`, operatori `%`.

Përkufizim

Python ofron disa mënyra për të aksesuar karakteret individualë brenda një **string**-u. Stringjet kanë gjithashtu *metoda* që të lejojnë të kryesh operacione mbi to.

Shumë nga programet që ke shkruar deri tani kanë punuar me stringje, por vetëm në një mënyrë të kufizuar. Operacionet që ke kryer me stringje deri tani kanë përfshirë kryesisht vetëm *hyrje* dhe *dalje*. Për shembull, ke lexuar stringje si input nga tastiera dhe nga skedarët, dhe ke dërguar stringje si output në ekran dhe në skedarë. Ka shumë lloje programesh që jo vetëm lexojnë stringje si input dhe shkruajnë stringje si output, por gjithashtu kryejnë operacione mbi stringje. Programet e *përpunimit të tekstit*, për shembull, manipulojnë sasi të mëdha teksti, dhe për rrjedhojë punojnë gjerësisht me stringje. Programet e email-it dhe motorët e kërkimit janë shembuj të tjerë programesh që kryejnë operacione mbi stringje. Python ofron një gamë të gjerë mjetesh dhe teknikash programimi që mund t'i përdorësh për të ekzaminuar dhe manipuluar stringje. Në fakt, stringjet janë një *lloj sekuence*, prandaj shumë nga konceptet që mësove për sekuencat në kapitujt e mëparshëm vlejnë edhe për stringjet. Në këtë kapitull do të shohim shumë prej tyre.

1.1 Aksesimi i karaktereve individualë në një string

Disa detyra programimi kërkojnë që të aksesosh karakterët individualë në një string. Për shembull, me siguri je i njohur me faqet e uebit që të kërkojnë të vendosësh një fjalëkalim. Për arsye sigurie, shumë sajte kërkojnë që fjalëkalimi yt të ketë të paktën një shkronjë *me të madhe*, të paktën një shkronjë *me të vogël*, dhe të paktën një *shifër*.

Kur ti vendos fjalëkalimin, një program ekzaminon secilin karakter për t'u siguruar që fjalëkalimi i plotëson këto kushte. (Më vonë në këtë kapitull do të shohësh një shembull programi që bën pikërisht këtë.) Në këtë seksion do të shohim dy teknika që mund të përdorësh në Python për të aksesuar karakterët individualë të një stringu: përdorimi i ciklit `for` dhe **indeksimi**.

1.1.1 Iterimi mbi një string me ciklin `for`

Një nga mënyrat më të lehta për të aksesuar karakterët individualë në një string është të përdorësh ciklin `for`. Ja formati i përgjithshëm:

```
for variable in string:
    statement
    statement
```

Në këtë format të përgjithshëm, `variable` është emri i një variableje, dhe `string` është ose një *literal* string, ose një variabël që referon një string. Sa herë që cikli iteron, `variable` do të referojë një *kopje* të një karakteri në `string`, duke filluar nga karakteri i parë. Themi se cikli *iteron mbi karakterët* e stringut. Ja një shembull:

```
name = 'Juliet'
for ch in name:
    print(ch)
```

Variabla `name` referon një string me gjashtë karaktere, ndaj ky cikël do të iteron gjashtë herë. Herën e parë që cikli iteron, variabla `ch` do të referojë `'J'`, herën e dytë `'u'`, dhe kështu me radhë. Kjo ilustrohet në Fig. 1.1. Kur kodi ekzekutohet, do të shfaqet:

```
J
u
l
i
e
t
```

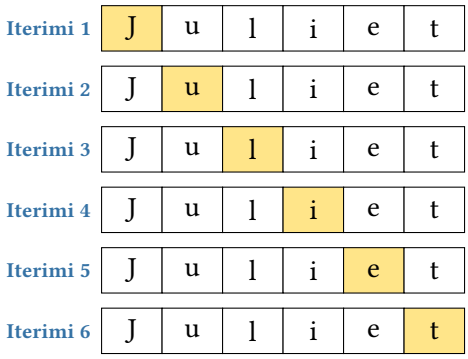

Figure 1.1: Iterimi mbi stringun `'Juliet'`: në çdo hap `ch` merr nga një karakter me radhë.

Shënim

Figura 1.1 ilustron si `ch` referon një *kopje* të një karakteri nga stringu, ndërsa cikli iteron. Nëse e ndryshojmë vlerën që `ch` referon brenda ciklit, kjo **nuk** ka efekt mbi stringun që referon `name`. Për shembull, shiko kodin më poshtë: rreshti 3 thjesht i ricakton `ch` një vlerë tjetër në çdo iterim; nuk ndikon te `'Juliet'` dhe as te numri i iterimeve.

```
1 name = 'Juliet'
2 for ch in name:
3     ch = 'X'
4 print(name)
```

Kur ky kod ekzekutohet, do të printohet:

```
Juliet
```

Programi vijues tregon një shembull tjetër. Ai i kërkon përdoruesit të shkruajë një string dhe pastaj përdor një `for`-cikël për të *numëruar* sa herë shfaqet shkronja `T` (me të madhe ose me të vogël).

Programi 1 (`count_Ts.py`)

```
def main():
    count = 0
    my_string = input('Shkruaj një fjali: ')

    for ch in my_string:
        if ch == 'T' or ch == 't':
            count += 1

    print(f'Shkronja T shfaqet {count} herë.')

if __name__ == '__main__':
    main()
```

Kur ky program ekzekutohet, ai mund të sillet kështu:

```
Shkruaj një fjali: Today we sold twenty-two toys.
Shkronja T shfaqet 5 herë.
```

1.2 Indeksimi

Një mënyrë tjetër për të aksesuar karakterët individualë në një string është me **indeks**. Çdo karakter në një string ka një indeks që përcakton pozicionin e tij në string. Indeksimi fillon nga `0`, pra indeksi i karakterit të parë është `0`, i dyti `1`, e kështu me radhë. Indeksi i karakterit të fundit është `len(string) - 1`. Figura 1.2 tregon indekset për çdo karakter në stringun `'Roses are red'`. Stringu ka 13 karaktere, ndaj indekset vlefshme janë `0` deri `12`.

Mund të përdorësh një indeks për të marrë një *kopje* të një karakteri individual nga stringu, si më poshtë:

R	o	s	e	s	␣	a	r	e	␣	r	e	d
0	1	2	3	4	5	6	7	8	9	10	11	12

Figure 1.2: Indekset e karaktereve për `'Roses are red'`. Hapësira paraqitet si ␣.

```
my_string = 'Roses are red'
ch = my_string[6]
```

Shprehja `my_string[6]` në deklaratën e dytë kthen një kopje të karakterit në indeksin `6` të `my_string`. Pas ekzekutimit, `ch` do të referojë `'a'`, siç tregohet në Fig. 1.3.

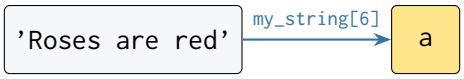

Figure 1.3: Marrja e një kopjeje karakteri nga stringu.

Ja një shembull tjetër:

```
my_string = 'Roses are red'
print(my_string[0], my_string[6], my_string[10])
```

```
R a r
```

Mund të përdorësh edhe **indekse negative**, për të specifikuar pozicione në raport me fundin e stringut. Interpretuesi i Python-it i *shton* indekset negative me gjatësinë e stringut për të gjetur pozicionin e saktë. Indeksi `-1` identifikon karakterin e fundit, `-2` parafunditin, e kështu me radhë:

```
my_string = 'Roses are red'
print(my_string[-1], my_string[-2], my_string[-13])
```

```
d e R
```

-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
R	o	s	e	s	␣	a	r	e	␣	r	e	d
0	1	2	3	4	5	6	7	8	9	10	11	12

Indekset negative llogariten nga fundi: -1 është karakteri i fundit, -13 është i pari.

Figure 1.4: Indekset negative për `'Roses are red'`.

1.2.1 Përjashtimet `IndexError`

Një përjashtim `IndexError` do të ndodhë nëse përpiqesh të përdorësh një indeks që është jashtë kufijve për një string të caktuar. Për shembull, stringu `'Boston'` ka 6 karaktere, ndaj indekset pozitive të vlefshme janë `0` deri `5` (indekset negative të vlefshme janë `-1` deri `-6`). Kodi vijues shkakton `IndexError`:

```
city = 'Boston'
print(city[6])
```

Ky lloj gabimi zakonisht ndodh kur një `loop` iteron gabimisht përtej fundit të stringut, si këtu:

```
city = 'Boston'
index = 0
while index < 7:
    print(city[index])
    index += 1
```

Herën e fundit që ky `loop` iteron, `index` merr vlerën `6`, e cila është indeks i pavlefshëm për `'Boston'`. Si pasojë, `print(city[index])` do të ngrejë `IndexError`.

1.3 Funksioni `len`

Në kapitullin për sekuencat mësove për funksionin `len`, i cili kthen gjatësinë e një sekuence. `len` përdoret edhe për të marrë gjatësinë e një stringu:

```
city = 'Boston'
size = len(city)
```

Deklarata e dytë thërret `len`, duke i kaluar `city` si argument. Funksioni kthen vlerën `6`, që është gjatësia e `'Boston'`, dhe ajo i caktohet `size`. `len` është veçanërisht i dobishëm për të **parandaluar** që `loop`-et të kalojnë përtej fundit të një stringu:

```
city = 'Boston'
index = 0
while index < len(city):
    print(city[index])
    index += 1
```

Vër re që `loop`-i vazhdon sa kohë `index` është më i vogël se gjatësia e stringut, sepse indeksi i karakterit të fundit është gjithmonë `len(string) - 1`.

1.4 Konkatenimi i stringjeve

Një operacion i zakonshëm mbi stringje është **konkatenimi**, pra ngjitja e një stringu në fund të një tjetri. Ke parë shembuj që përdorin operatorin `+` për të konkatenur stringje: ai prodhon një string që është kombinimi i dy operandëve të tij.

```
1 message = 'Hello ' + 'world'
2 print(message)
```

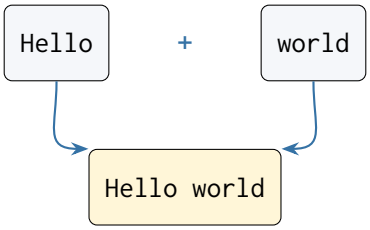

Figure 1.5: Konkatenimi i dy stringjeve për të formuar një string të ri.

Rreshti 1 konkatenon `'Hello '` dhe `'world'` për të prodhuar `'Hello world'`, i cili i caktohet `message`. Rreshti 2 printon stringun e referuar nga `message`. Të shikojmë një shembull tjetër:

```
1 first_name = 'Emily'
2 last_name = 'Yeager'
3 full_name = first_name + ' ' + last_name
4 print(full_name)
```

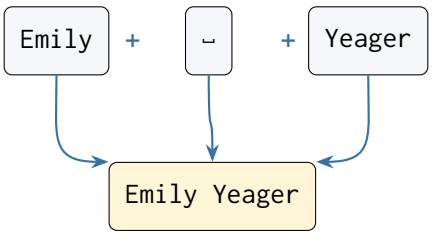

Figure 1.6: Konkatenimi i emrit dhe mbiemrit për të formuar `full_name`.

Mund të përdorësh edhe operatorin `+=` për konkatenim:

```
1 letters = 'abc'
2 letters += 'def'
3 print(letters)
```

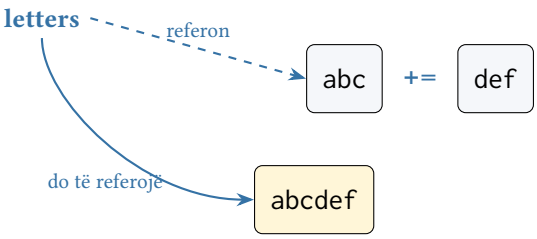

Figure 1.7: Ilustrim i përdorimit të operatorit `+=`

Deklarata në rreshtin 2 është e njëjtë me:

```
letters = letters + 'def'
```

Shembull shtesë:

```
name = 'Kelly' # name është 'Kelly'
name += ' ' # name është 'Kelly '
name += 'Yvonne' # name është 'Kelly Yvonne'
name += ' ' # name është 'Kelly Yvonne '
name += 'Smith' # name është 'Kelly Yvonne Smith'
print(name)
```

Kujdes

Operandi në anën e majtë të `+=` duhet të jetë *një variabël ekzistuese.* Nëse specifikon një variabël që nuk ekziston, do të ngrihet një përjashtim.

1.5 Stringjet janë *immutable*

Në Python, stringjet janë *immutable*, që do të thotë se pasi krijohen, **nuk** mund të ndryshohen. Disa operacione, si konkatenimi, mund të japin përshtypjen se modifikojnë stringun, por në të vërtetë krijojnë një string të *ri.*

Programi 2 (`concatenate.py`)

```
def main():
    name = 'Carmen'
    print(f'The name is: {name}')
    name = name + ' Brown'
    print(f'Now the name is: {name}')

if __name__ == '__main__':
    main()
```

Kur ky program ekzekutohet, do të shfaqet:

```
The name is: Carmen
Now the name is: Carmen Brown
```

Deklarata në rreshtin e parë brenda `main` i cakton `'Carmen'` variablës `name` (Fig. 1.8). Më pas, `name = name + ' Brown'` krijon *një string të ri* `'Carmen Brown'` dhe ia cakton sërish `name` (Fig. 1.9). Stringu origjinal `'Carmen'` *nuk* modifikohet; thjesht mbetet i pareferencuar dhe do të pastrohet nga interpretuesi kur të jetë e përshtatshme.

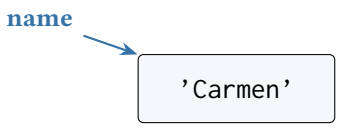

Figure 1.8: Stringu `'Carmen'` i caktuar te `name`.

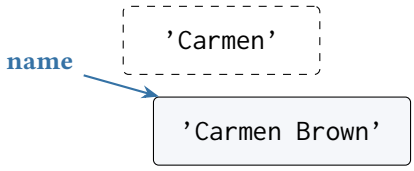

Figure 1.9: Pas `name = name + ' Brown'` krijohet një string i ri `'Carmen Brown'`.

Immutability dhe caktimi me indeks

Për shkak se stringjet janë *immutable*, nuk mund të përdorësh një shprehje të formës `string[index]` në anën e majtë të operatorit të caktimit. Shembulli më poshtë do të ngrejë një përjashtim, sepse përpiqet të ndryshojë vlerën e karakterit të parë të `'Bill'`:

```
friend = 'Bill'
friend[0] = 'J' # Jo —kjo shkakton gabim!
```

1.6 Prerja (*slicing*) e stringjeve

Përkufizim

Mund të përdorësh shprehje *slicing* për të zgjedhur një interval karakteresh nga një string.

Ke mësuar më parë se një *prerje* (slice) është një varg elementësh të marrë nga një sekuencë. Kur bën një prerje nga një string, merr një varg karakteresh nga brenda atij stringu. Prerjet e stringjeve quhen edhe *nënstringje* (*substrings*). Për të marrë një prerje nga një string, përdoret formati i përgjithshëm:

```
string[start : end]
```

Në këtë format, `start` është indeksi i karakterit të parë në prerje, dhe `end` është indeksi që shënon *fundin* e prerjes. Shprehja kthen një string që përmban një kopje të karaktereve nga `start` e deri *(por pa përfshirë)* `end`. Për shembull, supozojmë se kemi:

```
full_name = 'Patty Lynn Smith'
middle_name = full_name[6:10]
```

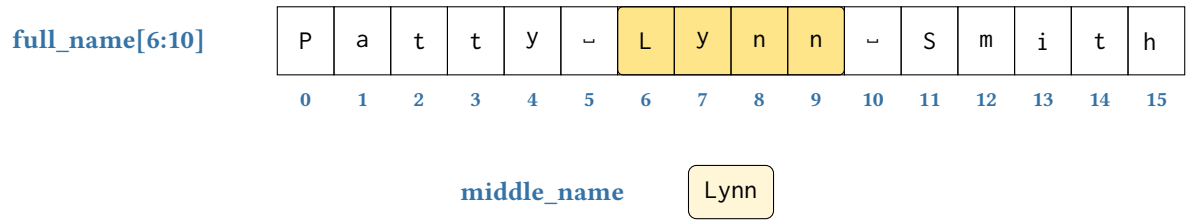

Figure 1.10: Prerja `full_name[6:10]` zgjedh karakteret me indekset 6–9, che formano `'Lynn'`.

Deklarata e dytë i cakton variablës `middle_name` stringun `'Lynn'`. Nëse e **lë bosh indeksin `start`**, Python përdor `0` si

indeks fillestar. Shembull:

```
full_name = 'Patty Lynn Smith'
first_name = full_name[:5]
```

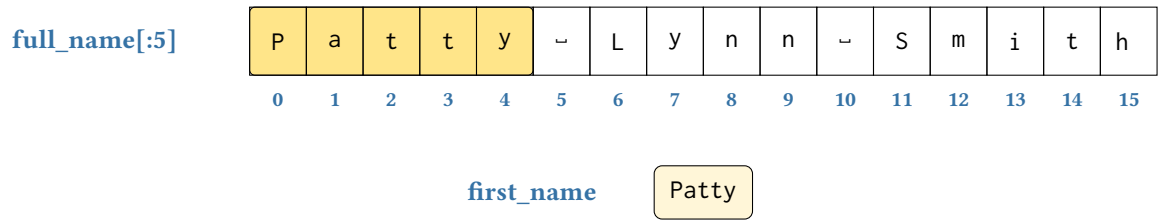

Figure 1.11: Prerja `full_name[:5]` zgjedh karakteret me indekset 0–4, që formojnë `'Patty'`.

Deklarata e dytë i cakton `first_name` stringun `'Patty'`. Nëse e **lë bosh indeksin `end`**, Python përdor gjatësinë e stringut si indeks fundor. Shembull:

```
full_name = 'Patty Lynn Smith'
last_name = full_name[11:]
```

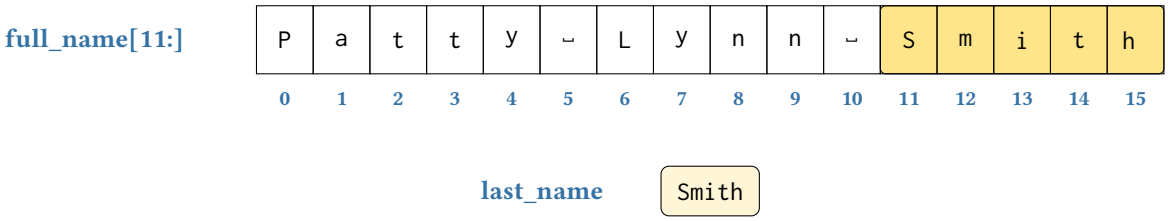

Figure 1.12: Prerja `full_name[11:]` zgjedh karakteret me indekset 11–15, që formojnë `'Smith'`.

Deklarata e dytë i cakton `last_name` stringun `'Smith'`. Çfarë mendon se i cakton kodi i mëposhtëm variablës `my_string`?

```
full_name = 'Patty Lynn Smith'
my_string = full_name[:]
```

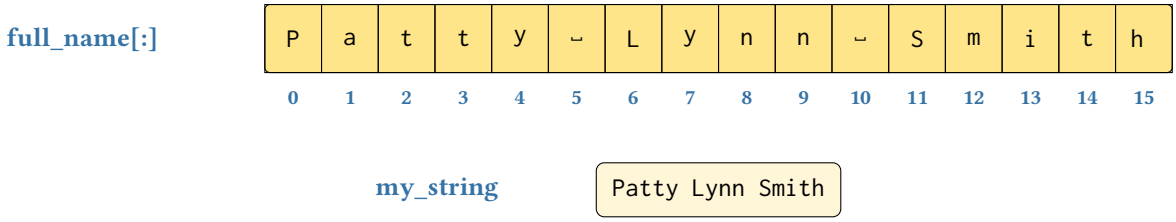

Figure 1.13: Prerja `full_name[:]` kthen të gjithë stringun.

Deklarata e dytë i caktoi `my_string` të gjithë stringun `'Patty Lynn Smith'`. Kjo është e barasvlershme me:

```
my_string = full_name[0 : len(full_name)]
```

Prerjet që pamë deri tani marrin vargje *të njëpasnjëshme* karakteresh. Shprehjet *slicing* mund të kenë edhe një **hap** (*step*), i cili bën që të kapërcehen karaktere në string. Ja një shembull me vlerë hapi:

```
letters = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
print(letters[0:26:2])
```

Numri i tretë në kllapa është vlera e *hapit*. Një hap `2`, si në shembullin më sipër, bën që prerja të përmbajë çdo të dytin karakter nga intervali i specifikuar në string. Ky kod do të printojë:

```
ACEGIKMOQSUWY
```

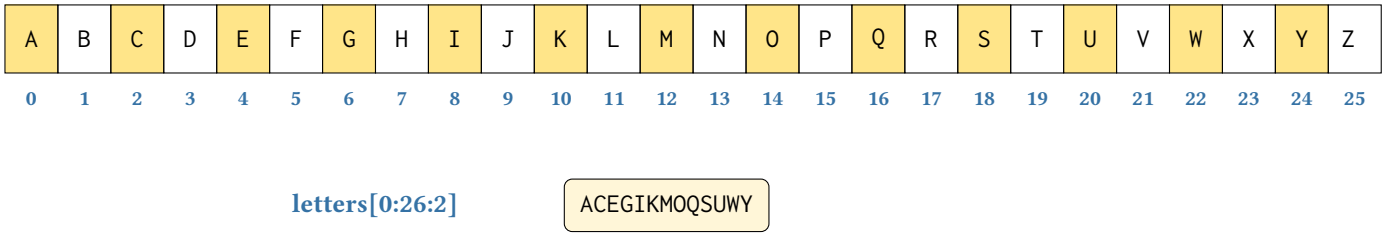

Figure 1.14: Prerja me hap 2 zgjedh çdo të dytin karakter: `letters[0:26:2]` kthen `ACEGIKMOQSUWY`.

Mund të përdorësh edhe **indekse negative** në shprehjet e *slicing* për të referuar pozicione në raport me fundin e stringut. Shembull:

```
full_name = 'Patty Lynn Smith'
last_name = full_name[-5:]
```

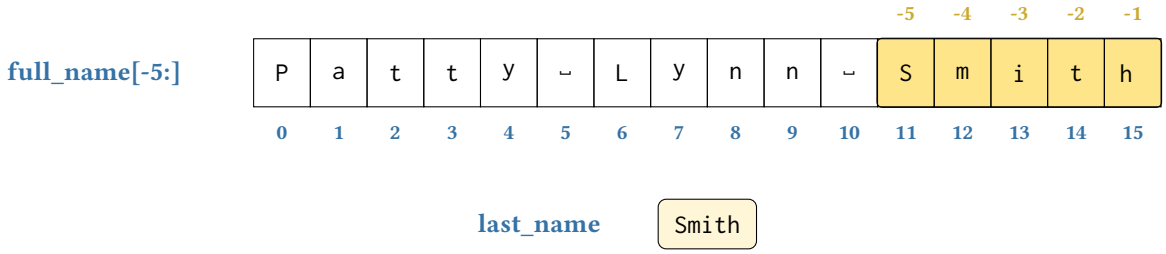

Figure 1.15: Prerja `full_name[-5:]` zgjedh pesë karakteret e fundit dhe kthen `Smith`.

Kujto që Python i *shton* indekset negative me gjatësinë e stringut për të gjetur pozicionin që referohet nga ai indeks. Deklarata e dytë më sipër i cakton `last_name` stringun `'Smith'`.

Shënim: indekse të pavlefshme në *slicing*

Indekset e pavlefshme **nuk** bëjnë që shprehjet *slicing* të ngrejnë përjashtim.

- Nëse `end` shënon një pozicion përtej fundit të stringut, Python përdor gjatësinë e stringut në vend të tij.
- Nëse `start` shënon një pozicion përpara fillimit të stringut, Python përdor `0` në vend të tij.
- Nëse `start` është më i madh se `end`, shprehja *slicing* kthen një string bosh.

1.7 Testimi, kërkimi dhe manipulimi i stringjeve

Përkufizim

Python ofron *operatorë* dhe *metoda* për të testuar stringje, për të kërkuar brendinë e tyre dhe për të marrë kopje të modifikuara të stringjeve.

1.7.1 Testimi i stringjeve me `in` dhe `not in`

Në Python, mund të përdorësh operatorin `in` për të përcaktuar nëse një string përmbahet brenda një tjetri. Formati i përgjithshëm:

```
string1 in string2
```

`string1` dhe `string2` mund të jenë literale ose variabla që referojnë stringje. Shprehja kthen `True` nëse `string1` gjendet brenda `string2`. Shembull:

```
text = 'Four score and seven years ago'
if 'seven' in text:
    print('The string "seven" was found.')
else:
    print('The string "seven" was not found.')
```

Kur ekzekutohet, ky kod do të printojë:

```
The string "seven" was found.
```

Operatori `not in` përcakton nëse një string *nuk* përmbahet në një tjetër:

```
names = 'Bill Joanne Susan Chris Juan Katie'
if 'Pierre' not in names:
    print('Pierre was not found.')
else:
    print('Pierre was found.')
```

Kur ekzekutohet, ky kod do të printojë:

```
Pierre was not found.
```

1.7.2 Metoda testimi për stringje

Siç mësove më parë, një *metodë* është një funksion që i përket një objekti dhe kryen një operacion mbi të. Stringjet në Python kanë shumë metoda.[1] Në këtë seksion do të shohim disa metoda për:

- testimin e vlerave të stringjeve;

[1] Lista e plotë e metodave të stringjeve: dokumentacioni i Python tek https://www.python.org.

- kryerjen e modifikimeve të ndryshme;
- kërkimin e nënstringjeve dhe zëvendësimin e sekuencave të karaktereve.

Formati i përgjithshëm i një thirrjeje metode është:

```
stringvar.method(arguments)
```

Metodat në Tab. 1.1 testojnë një string për karakteristika specifike.

Metoda	Përshkrimi
`isalnum()`	`True` nëse stringu ka vetëm shkronja/digjitë *dhe* ka të paktën 1 karakter; përndryshe `False`.
`isalpha()`	`True` nëse stringu ka vetëm shkronja dhe ka të paktën 1 karakter; ndryshe `False`.
`isdigit()`	`True` nëse stringu ka vetëm shifra dhe ka të paktën 1 karakter; ndryshe `False`.
`islower()`	`True` nëse të gjitha shkronjat alfabetike janë me të vogla *dhe* ka të paktën një shkronjë; ndryshe `False`.
`isspace()`	`True` nëse stringu ka vetëm hapësira të bardha (space, `\n`, `\t`) dhe të paktën 1 karakter; ndryshe `False`.
`isupper()`	`True` nëse të gjitha shkronjat alfabetike janë me të mëdha *dhe* ka të paktën një shkronjë; ndryshe `False`.

Table 1.1: Disa metoda testimi për stringje.

Shembuj të metodave të testimit të stringjeve

```
string1 = '1200'
if string1.isdigit():
    print(f'{string1} contains only digits.')
else:
    print(f'{string1} contains characters other than digits.')
```

```
1200 contains only digits.
```

```
string2 = '123abc'
if string2.isdigit():
    print(f'{string2} contains only digits.')
else:
    print(f'{string2} contains characters other than digits.')
```

```
123abc contains characters other than digits.
```

Programi 3 (`string_test.py`)

```
def main():
    # Get a string from the user.
    user_string = input('Enter a string: ')

    print('This is what I found about that string:')

```

```
    # Test the string.
    if user_string.isalnum():
        print('The string is alphanumeric.')
    if user_string.isdigit():
        print('The string contains only digits.')
    if user_string.isalpha():
        print('The string contains only alphabetic characters.')
    if user_string.isspace():
        print('The string contains only whitespace characters.')
    if user_string.islower():
        print('The letters in the string are all lowercase.')
    if user_string.isupper():
        print('The letters in the string are all uppercase.')

if __name__ == '__main__':
    main()
```

Shembuj të programit të testimit të stringjeve

Shembulli i mëposhtëm tregon një ekzekutim të programit `string_test.py` ku përdoruesi jep një string me vetëm shkronja të vogla:

```
Enter a string: abc
This is what I found about that string:
The string is alphanumeric.
The string contains only alphabetic characters.
The letters in the string are all lowercase.
```

Shembulli i mëposhtëm tregon një ekzekutim të programit `string_test.py` ku përdoruesi jep një string me vetëm digjite:

```
Enter a string: 123
This is what I found about that string:
The string is alphanumeric.
The string contains only digits.
```

Shembulli i mëposhtëm tregon një ekzekutim të programit `string_test.py` ku përdoruesi jep një string me numra dhe shkronja të mëdha:

```
Enter a string: 123ABC
This is what I found about that string:
The string is alphanumeric.
The letters in the string are all uppercase.
```

1.7.3 Metoda modifikimi (kthejnë kopje të modifikuara)

Edhe pse stringjet janë *immutable*, ato kanë metoda që kthejnë *kopje* të modifikuara. Disa prej tyre janë listuar në Tab. 1.2.

Shembull `lower()`:

Metoda	Përshkrimi
`lower()`	Kthen një kopje ku të gjitha shkronjat alfabetike janë kthyer në të vogla.
`lstrip()`	Kthen një kopje pa hapësirat e bardha *në fillim* të stringut (space, `\n`, `\t`).
`lstrip(char)`	Heq të gjitha instancat e `char` në fillim të stringut dhe kthen kopjen.
`rstrip()`	Kthen një kopje pa hapësirat e bardha *në fund* të stringut.
`rstrip(char)`	Heq të gjitha instancat e `char` në fund të stringut dhe kthen kopjen.
`strip()`	Kthen një kopje pa hapësirat e bardha në fillim *dhe* në fund.
`strip(char)`	Heq të gjitha instancat e `char` në fillim dhe fund dhe kthen kopjen.
`upper()`	Kthen një kopje ku të gjitha shkronjat alfabetike janë kthyer në të mëdha.

Table 1.2: Metoda të zakonshme modifikimi për stringje.

```
letters = 'WXYZ'
print(letters, letters.lower())
```

Kur ekzekutohet, ky kod do të printojë:

```
WXYZ wxyz
```

Shembull `upper()`:

```
letters = 'abcd'
print(letters, letters.upper())
```

Kur ekzekutohet, ky kod do të printojë:

```
abcd ABCD
```

`lower` dhe `upper` janë të dobishme për krahasime *jo-ndjeshme ndaj shkronjave të mëdha/vogla* (case-insensitive). Krahasimet e zakonshme janë *case-sensitive*. Shembull:

```
again = 'y'
while again.lower() == 'y':
    print('Hello')
    print('Do you want to see that again?')
    again = input('y = yes, anything else = no: ')
```

Rezultat i ngjashëm duke përdorur `upper()`:

```
again = 'y'
while again.upper() == 'Y':
    print('Hello')
    print('Do you want to see that again?')
    again = input('y = yes, anything else = no: ')
```

1.7.4 Kërkimi dhe zëvendësimi

Programet shpesh kanë nevojë të kërkojnë nënstringje (stringje që shfaqen brenda stringjeve të tjera). Disa metoda të dobishme janë listuar në Tab. 1.3.

Metoda	Përshkrimi
`endswith(substring)`	Kthen `True` nëse stringu përfundon me `substring`.
`find(substring)`	Kthen indeksin më të vogël ku gjendet `substring`; `-1` nëse nuk gjendet.
`replace(old, new)`	Kthen një kopje ku çdo paraqitje e `old` zëvendësohet me `new`.
`startswith(substring)`	Kthen `True` nëse stringu fillon me `substring`.

Table 1.3: Metoda për kërkim dhe zëvendësim te stringjet.

Shembull `endswith` / `startswith`:

```
filename = input('Enter the filename: ')
if filename.endswith('.txt'):
    print('That is the name of a text file.')
elif filename.endswith('.py'):
    print('That is the name of a Python source file.')
elif filename.endswith('.doc'):
    print('That is the name of a word processing document.')
else:
    print('Unknown file type.')
```

`find` kthen indeksin e parë të nënstringut, ose `-1` nëse nuk gjendet:

```
string = 'Four score and seven years ago'
position = string.find('seven')
if position != -1:
    print(f'The word "seven" was found at index {position}.')
else:
    print('The word "seven" was not found.')
```

Kur ekzekutohet, ky kod do të printojë:

```
The word "seven" was found at index 15.
```

`replace` kthen një kopje me zëvendësime:

```
string = 'Four score and seven years ago'
new_string = string.replace('years', 'days')
print(new_string)
```

Kur ekzekutohet, ky kod do të printojë:

```
Four score and seven days ago
```

1.8 Operatori i përsëritjes (*)

Përkufizim

Operatori i përsëritjes `*` krijon një string të ri që përmban `n` kopje të një stringu të dhënë. Kjo është një mënyrë e thjeshtë për të gjeneruar përsëritje të një modeli teksti (p.sh. rradhë me karaktere për vizatime të thjeshta).

Rregullat kryesore që duhet mbajtur mend:

- Vlera `n` duhet të jetë një numër i plotë (int). Nëse jep lloj tjetër (p.sh. float), Python hedh `TypeError`.
- Nëse `n` është 0 ose negative, rezultati është string bosh `''`.
- Operatori nuk ndryshon stringun original (stringjet janë *immutable*); ai kthen një string të ri.

Formati i përgjithshëm:

```
string_to_copy * n
```

Shembuj të thjeshtë:

```
my_string = 'w' * 5
print(my_string) # wwwww

print('Hello' * 3) # HelloHelloHello

print('A' * 0) # '' (string bosh)
print('B' * -2) # '' (string bosh)
```

Përdorime të zakonshme:

- Krijimi i separatorëve vizualë (p.sh. `'-' * 40` për një vijë).
- Ndërtimi i pamjeve të thjeshta ASCII (p.sh. trekëndësha me `'*'`).
- Shpejtimi i prodhimit të një teksti të përsëritur pa cikël.

Programi 4 (`repetition_operator.py`) — shembull i pasuruar

```
def main():
    # Print nine rows increasing in length.
    for count in range(1, 10):
        print('Z' * count)

    # Print nine rows decreasing in length.
    for count in range(8, 0, -1):
        print('Z' * count)

    # Separator i përdorur në raporte
    print('-' * 40)
    print('Report'.center(40))
    print('-' * 40)

if __name__ == '__main__':
```

```
    main()
```

1.9 Ndarja e një stringu me `split()`

Përkufizim

Metoda `split()` kthen një *listë* me pjesët e stringut të ndara sipas një *ndarësi* (separator). Nëse nuk i kalon asnjë argument, `split()` ndan sipas hapësirave të bardha dhe bashkon grupet e hapësirave (pra shumë hapësira radhazi trajtohen si një ndarës).

Pikat kryesore:

- `string.split()` (pa argument) ndan sipas çdo lloj hapësire (space, `\n`, `\t`) dhe: heq hapësirat në fillim/fund, dhe grupet e shumëfishta hapësirash konsiderohen si një ndarës i vetëm.
- `string.split(sep)` me një `sep` të përcaktuar ndan në çdo shfaqje të saktë të `sep`. Në këtë rast, grupet e ngjashme të `sep` *nuk* bashkohen dhe hapësirat rreth pjesëve nuk hiqen automatikisht.
- Mund të vendosësh një numër maksimal ndarjesh me argumentin opsional `maxsplit`: `string.split(sep, maxsplit)`.
- Nuk mund të përdorësh string bosh si `sep` (kjo jep `ValueError`).

Shembuj bazë:

```python
my_string = 'One two three\tfour\nfive'
print(my_string.split()) # ['One', 'two', 'three', 'four', 'five']

csv = 'a, b, c, d'
print(csv.split(',')) # ['a', ' b', ' c', ' d'] (vini re hapësirat)

print('word1 word2 word3'.split(maxsplit=1)) # ['word1', 'word2 word3']
```

Këshillë praktike

Kur ndan tekste si CSV ose lista me elemente të ndara me presje, shpesh do të dëshironi të pastroni elementët e listës me `strip()` për të hequr hapësirat përpara dhe pas:

```python
parts = [p.strip() for p in csv.split(',')]
print(parts) # ['a', 'b', 'c', 'd']
```

Program i vogël ilustrues: lexon një fjali dhe numëron sa herë shfaqet secila fjalë (thjesht për shembull, pa trajtuar pikësimin):

Programi 6 (`word_count.py`)

```python
def main():
    text = input('Shkruaj një fjali: ')
    words = text.split() # ndan sipas hapësirave
    counts = {}
```

```python
    for w in words:
        w = w.lower() # krahasim jo-case-sensitive
        counts[w] = counts.get(w, 0) + 1

    print('Numri i fjalëve:')
    for w, c in counts.items():
        print(f'{w}: {c}')

if __name__ == '__main__':
    main()
```

Këto koncepte (`split()`, `strip()`, dhe manipulimi i listes) janë shumë të dobishme për pastrimin/parsing-un e tekstit dhe për ndërtimin e programeve që përpunojnë fjalë, data ose skedarë të formatuar.

1.10 Karakteret e *escape*-it në stringje

Përkufizim

Një *escape character* është një `\` e ndjekur nga një karakter i veçantë, që lejon futjen në string të shenjave që përndryshe do ishin të paligjshme (p.sh. thonjëza brenda thonjëzave).

1.10.1 Pse duhen karakteret e *escape*-it

Shembulli më poshtë **shkakton gabim**, sepse `"` brenda stringut mbyll literalin:

```python
txt = "We are the so-called "Vikings" from the north."
```

Zgjidhja është të *ikin* thonjëzat e brendshme me `\"`:

```python
txt = "We are the so-called \"Vikings\" from the north."
print(txt)
```

1.10.2 Shpesh të përdorura

Kodi	Përshkrimi
`\'`	Thonjëz e vetme brenda '...'
`\\`	Vetë backslash
`\n`	Resht i ri (Newline)
`\r`	Carriage return
`\t`	Tab
`\b`	Backspace
`\f`	Form feed
`\ooo`	Vlerë oktale (p.sh. `\141` = 'a')
`\xhh`	Vlerë heksadecimale (p.sh. `\x61` = 'a')

Table 1.4: Karaktere *escape* të zakonshme në Python.

1.10.3 Shembuj të shpejtë

```
print("Line1\nLine2") # rresht i ri
print("Col1\tCol2") # tab
print("Backslash: \\") # shfaq "\"
print('It\'s fine!') # thonjëza e vetme brenda '...'
```

Raw strings dhe triple quotes

- **Raw strings** nuk interpretojnë *escape*-et: `r"C:\Users \Alion \Desktop \file .txt"`. Kujdes: s'mund të përfundojnë me një `\` të vetme.
- **Triple quotes** (`"'...'"`, `"""..."""`) lejojnë tekst shumë-rreshtor pa pasur nevojë për `\n`.

1.11 Formatimi i stringjeve

Python ofron disa mënyra për të futur vlera në një string.

1.11.1 F-strings (rekomanduar)

Sintaksë e thjeshtë, e lexueshme dhe e shpejtë (**Python 3.6+**):

```
name = "Alice"
age = 22
print(f"Name: {name}, Age: {age}")
```

```
Name: Alice, Age: 22
```

Mund të përdorësh edhe specifikime formati pas `:`:

```
pi = 3.14159
n = 7
print(f"pi = {pi:.2f}") # 2 shifra pas presjes
print(f"{n:04d}") # zgjat me 0 derisa në 4 shifra
print(f"{12345:,}") # ndarje mijëshe (lokalizimi varion)
print(f"{0.375:.1%}") # përqindje
```

1.11.2 `str.format()`

```
s = "My name is {} and I am {} years old.".format("Alice", 22)
t = "x = {x}, y = {y}".format(x=10, y=20)
u = "r={0:.2f}, a={1:>6}".format(3.14159, 42) # precizion & justifikim
print(s); print(t); print(u)
```

1.11.3 Operatori i vjetër % (stil "printf")

```
print("My name is %s and weight is %d kg!" % ("Zara", 21))
```

Specifikues të zakonshëm konvertimi:

Simbol	Kuptimi
`%s`	string (përdor `str()`)
`%d`, `%i`	numër i plotë me shenjë (decimal)
`%f`	numër real me presje lëvizëse
`%e`, `%E`	notacion shkencor (`e`/`E`)
`%x`, `%X`	numër heksadecimal (shkronja të vogla/të mëdha)
`%o`	numër oktal
`%c`	karakter (`int` → `chr`)
`%g`, `%G`	më i shkurtri midis `%f` dhe `%e`/`%E`

Table 1.5: Specifikues konvertimi për operatorin %.

Këshillë

Në Python 3, `%u` trajtohet si `%d` dhe është trashëgimi; prefero f-strings ose `format()` për kod modern.

Përmbledhje e kapitullit — *cheat-sheet*

Tema	Sintaksë / Shembull (→ rezultat)
Indeksimi	`s[i]`, `s[-1]` (i fundit)
Prerja	`s[a:b]`, `s[a:b:c]` p.sh. `'ABCDEF'[0:6:2] → 'ACE'`
Gjatësia	`len(s)`
Anëtarësia	`'py' in 'python' → True`, `'z' not in 'abc' → True`
Konkatenimi	`'Hello ' + 'world' → 'Hello world'`, `name += ' ' + surname`
Immutability	`s[i] = 'X'` **gabim**; përdor `s = s[:i] + 'X' + s[i+1:]`
Përsëritja	`'*' * 5 → '*****'`, `'A' * 0 → ''`
Ndarja	`'a, b, c'.split(',') → ['a', ' b', ' c']`
Pastrimi elementëve	`[p.strip() for p in 'a, b'.split(',')] → ['a','b']`
Kërkimi	`s.find('x') → indeks ose -1`, `s.startswith('pre')`, `s.endswith('.py')`
Zëvendësimi	`s.replace('old','new')`
Ndryshime të shpejta	`s.lower()`, `s.upper()`, `s.strip()`, `s.lstrip()`, `s.rstrip()`
Escape characters	`"Line1\nLine2"`, `"Col1\tCol2"`, `"\\"`
Raw/Triple strings	`r"C:\Users\Alion"`, `"""tekst shumë-rreshtor"""`
f-strings	`f"{name} ka {age} vjeç"`, `f"{pi:.2f}" → "3.14"`, `f"{n:04d}" → "0007"`
`str.format()`	`"x={0:.1f}, y={y}".format(3.14, y=2)`
Operatori `%`	`"%s %d" % ('Zara', 21)`; specifikues: `%s`, `%d`, `%f`, `%x`, `%e`, `%g`